Q1

```java
import java.util.*;
public class CourseRegistration {
    public static void main(String[] args) {
        Portal p = new Portal();
        p.addCourse(new Course("CS101", "Programming", 2));
        p.addCourse(new Course("CS201", "DSA", 2, "CS101"));

        p.addCompleted("S1", "CS101");
        p.addCompleted("S2", "CS101");
        p.addCompleted("S3", "CS101");

        p.register("S1", "CS201");
        p.register("S2", "CS201");
        p.register("S3", "CS201");
        p.register("S4", "CS201");
        p.register("S1", "CS201");
        p.show("CS201");

        p.drop("S1", "CS201");
        p.show("CS201");

        p.drop("S3", "CS201");
        p.show("CS201");
    }
}
class Course {
    String code;
    String name;
```

```
    int capacity;
    HashSet<String> enrolled = new HashSet<>();
    ArrayList<String> prerequisites = new ArrayList<>();
    LinkedList<String> waitlist = new LinkedList<>();

    Course(String code, String name, int capacity, String... prereqs) {
        this.code = code;
        this.name = name;
        this.capacity = capacity;
        prerequisites.addAll(Arrays.asList(prereqs));
    }
}

class Portal {
    HashMap<String, Course> courses = new HashMap<>();
    HashMap<String, HashSet<String>> completed = new HashMap<>();

    void addCourse(Course c) {
        courses.put(c.code, c);
    }

    void addCompleted(String studentId, String courseCode) {
        completed.computeIfAbsent(studentId, k -> new HashSet<>
()).add(courseCode);
    }

    void register(String studentId, String courseCode) {
```

```
    void register(String studentId, String courseCode) {
        Course c = courses.get(courseCode);
        if (c == null) {
            System.out.println("Course not found: " + courseCode);
            return;
        }
        if (c.enrolled.contains(studentId)) {
            System.out.println(studentId + " already enrolled in " +
courseCode);
            return;
        }
        if (c.waitlist.contains(studentId)) {
            System.out.println(studentId + " already on waitlist of " +
courseCode);
            return;
        }
        HashSet<String> done = completed.getOrDefault(studentId, new
HashSet<>());
        for (String p : c.prerequisites) {
            if (!done.contains(p)) {
                System.out.println(studentId + " missing prerequisite " + p
+ " for " + courseCode);
                return;
            }
        }
        if (c.enrolled.size() < c.capacity) {
            c.enrolled.add(studentId);
            System.out.println(studentId + " enrolled in " + courseCode);
```

```
        } else {
            c.waitlist.addLast(studentId);
            System.out.println(courseCode + " full. " + studentId + " added to waitlist at position " + c.waitlist.size());
        }
    }

    void drop(String studentId, String courseCode) {
        Course c = courses.get(courseCode);
        if (c == null) {
            System.out.println("Course not found: " + courseCode);
            return;
        }
        if (c.enrolled.remove(studentId)) {
            System.out.println(studentId + " dropped " + courseCode);
            if (!c.waitlist.isEmpty()) {
                String next = c.waitlist.removeFirst();
                c.enrolled.add(next);
                System.out.println(next + " promoted from waitlist to " + courseCode);
            }
        } else if (c.waitlist.remove(studentId)) {
            System.out.println(studentId + " removed from waitlist of " + courseCode);
        } else {
            System.out.println(studentId + " not in " + courseCode);
        }
    }
```

```
    }

    void show(String courseCode) {
        Course c = courses.get(courseCode);
        if (c == null) {
            System.out.println("Course not found: " + courseCode);
            return;
        }
        System.out.println(c.code + " " + c.name);
        System.out.println("Prerequisites: " + c.prerequisites);
        System.out.println("Enrolled (" + c.enrolled.size() + "/" + c.capacity + "): " + c.enrolled);
        System.out.println("Waitlist: " + c.waitlist);
    }
}

```

Output

```
S1 enrolled in CS201
S2 enrolled in CS201
CS201 full. S3 added to waitlist at position 1
S4 missing prerequisite CS101 for CS201
S1 already enrolled in CS201
CS201 DSA
Prerequisites: [CS101]
Enrolled (2/2): [S1, S2]
Waitlist: [S3]
S1 dropped CS201
S3 promoted from waitlist to CS201
CS201 DSA
Prerequisites: [CS101]
Enrolled (2/2): [S3, S2]
Waitlist: []
S3 dropped CS201
CS201 DSA
Prerequisites: [CS101]
Enrolled (1/2): [S2]
Waitlist: []
```

Q2

```
import java.util.*;
public class MediaPlayerQueue {
    public static void main(String[] args) {
        Player p = new Player();
        p.addLast(new Song("Sea", "BTS"));
        p.addLast(new Song("Beauty and a Beat", "Justin Bieber"));
        p.addLast(new Song("Cruel Summer", "Taylor Swift"));
        p.addLast(new Song("Confidence", "Kim"));
        p.addLast(new Song("Closer", "Chainsmokers"));
        p.addLast(new Song("Believer", "Imagine Dragons"));
        p.addLast(new Song("Cruel Summer", "Taylor Swift"));
        p.addFirst(new Song("Believer", "Imagine Dragons"));
        p.addAt(3, new Song("Sea", "BTS"));

        System.out.println("Forward:");
        p.showForward();

        p.removeDuplicates();
        System.out.println("After removing duplicates:");
        p.showForward();

        System.out.println("Backward:");
        p.showBackward();

        p.start();
        p.next();
        p.next();
        p.next();
```

```
        p.next();
        p.previous();
        p.previous();
        p.showHistory();
    }
}

class Song {
    String title;
    String artist;

    Song(String title, String artist) {
        this.title = title;
        this.artist = artist;
    }

    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Song)) return false;
        Song s = (Song) o;
        return title.equalsIgnoreCase(s.title) &&
artist.equalsIgnoreCase(s.artist);
    }

    public int hashCode() {
        return Objects.hash(title.toLowerCase(), artist.toLowerCase());
    }

    public String toString() {
        return title + " - " + artist;
    }
}

class Player {
    LinkedList<Song> queue = new LinkedList<>();
    LinkedList<Song> history = new LinkedList<>();
    ListIterator<Song> it;
    int maxHistory = 5;

    void addLast(Song s) {
        queue.addLast(s);
    }

    void addFirst(Song s) {
        queue.addFirst(s);
    }

    void addAt(int index, Song s) {
        if (index < 0 || index > queue.size()) {
            System.out.println("Invalid position");
            return;
        }
        queue.add(index, s);
    }
```

```
80      }
81
82      void removeDuplicates() {
83          HashSet<Song> seen = new HashSet<>();
84          Iterator<Song> i = queue.iterator();
85          while (i.hasNext()) {
86              if (!seen.add(i.next())) {
87                  i.remove();
88              }
89          }
90      }
91
92      void start() {
93          it = queue.listIterator();
94      }
95
96      void next() {
97          if (it == null) start();
98          if (it.hasNext()) {
99              Song s = it.next();
100             play(s);
101         } else {
102             System.out.println("End of queue");
103         }
104     }
105
106     void previous() {
107         if (it == null) start();
108         if (it.hasPrevious()) {
109             Song s = it.previous();
110             play(s);
111         } else {
112             System.out.println("Start of queue");
113         }
114     }
115
116     void play(Song s) {
117         System.out.println("Playing: " + s);
118         history.addFirst(s);
119         if (history.size() > maxHistory) {
120             history.removeLast();
121         }
122     }
123
124     void showForward() {
125         ListIterator<Song> li = queue.listIterator();
126         while (li.hasNext()) {
127             System.out.println(li.nextIndex() + ". " + li.next());
128         }
129     }
130
131     void showBackward() {
132         ListIterator<Song> li = queue.listIterator(queue.size());
133         while (li.hasPrevious()) {
134             System.out.println(li.previousIndex() + ". " + li.previous())
```

```
134                 System.out.println(li.previousIndex() + ". " + li.previous());
135             }
136         }
137 
138         void showHistory() {
139             System.out.println("Recent: " + history);
140         }
141     }
142 
```

Output

```
Forward:
0. Believer - Imagine Dragons
1. Sea - BTS
2. Beauty and a Beat - Justin Bieber
3. Sea - BTS
4. Cruel Summer - Taylor Swift
5. Confidence - Kim
6. Closer - Chainsmokers
7. Believer - Imagine Dragons
8. Cruel Summer - Taylor Swift
After removing duplicates:
0. Believer - Imagine Dragons
1. Sea - BTS
2. Beauty and a Beat - Justin Bieber
3. Cruel Summer - Taylor Swift
4. Confidence - Kim
5. Closer - Chainsmokers
Backward:
5. Closer - Chainsmokers
4. Confidence - Kim
3. Cruel Summer - Taylor Swift
2. Beauty and a Beat - Justin Bieber
1. Sea - BTS
0. Believer - Imagine Dragons
Playing: Believer - Imagine Dragons
Playing: Sea - BTS
Playing: Beauty and a Beat - Justin Bieber
Playing: Beauty and a Beat - Justin Bieber
Playing: Sea - BTS
Recent: [Sea - BTS, Beauty and a Beat - Justin Bieber, Beauty and a Beat - Justin Bieber, Sea - BTS, Believer - Imagine Dragons]
```